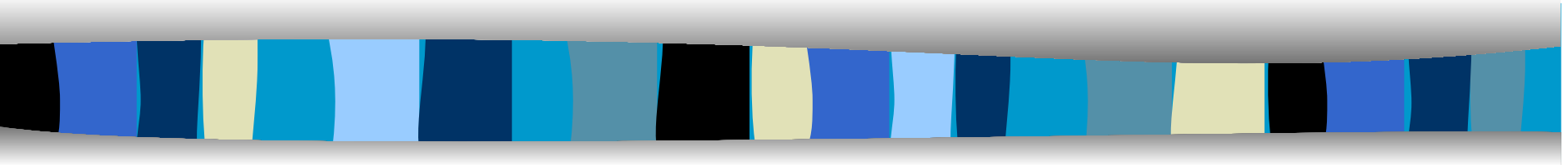
Cryptographic Hash Functions



MAC, MD5, SHA, Digital Signature

Hash Function

- A hash function H accepts a variable-length block of data M as input and produces a fixed-size hash value h = H(M).
- Here, h is hash value, H is hash function, and M is data or message
- Hash values is also known as hash code, or simply hash.
- A “good” hash function has the property that the results of applying the function to a large set of inputs will produce outputs that are evenly distributed and apparently random.

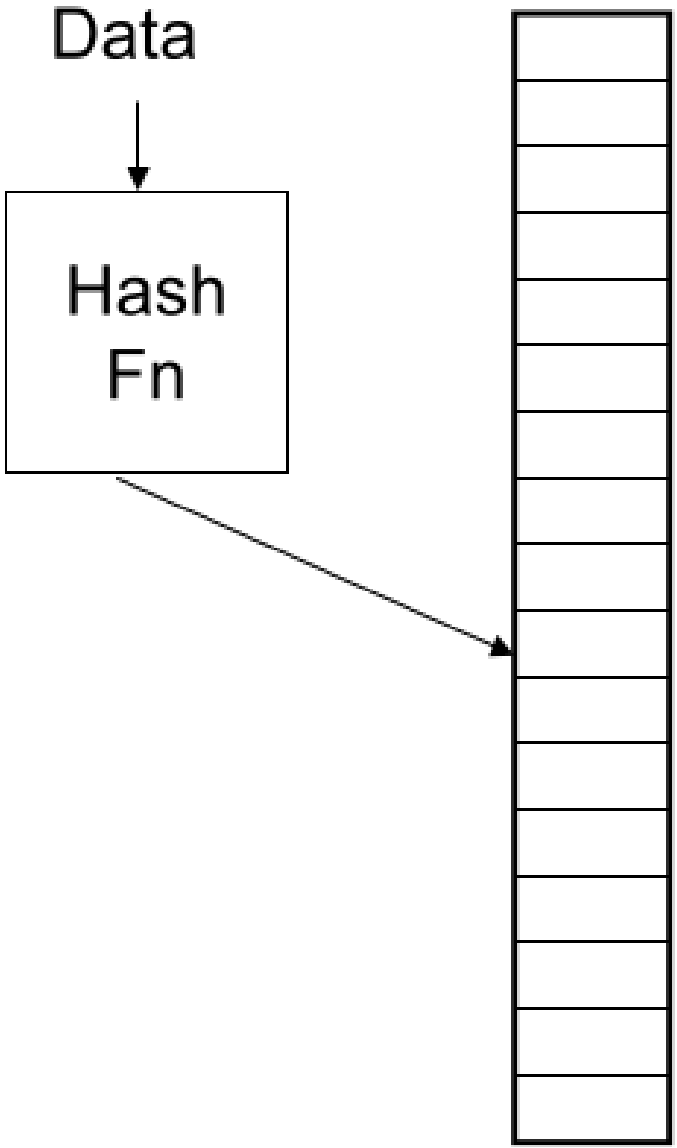
Hash Function

- The principal object of a hash function is data integrity.
- A change to any bit or bits in M results, with high probability, in a change to the hash code.
- h can be taken as the location or slot where M is stored.
- M can be taken as the key of hash function.
- Each location can store a single value.
- Set of all possible locations is called hash table.
- A good hash function uniformly distributes to slots.

Hash Function

- If there are n keys and m slots then
- The expected number of keys per slot assuming good hash function is
- a=n/m
- This is also called load factor.
- One simple hash function is
- h=H(k) mod m
- The size of k is variable and size of m is fixed.

Hash Function



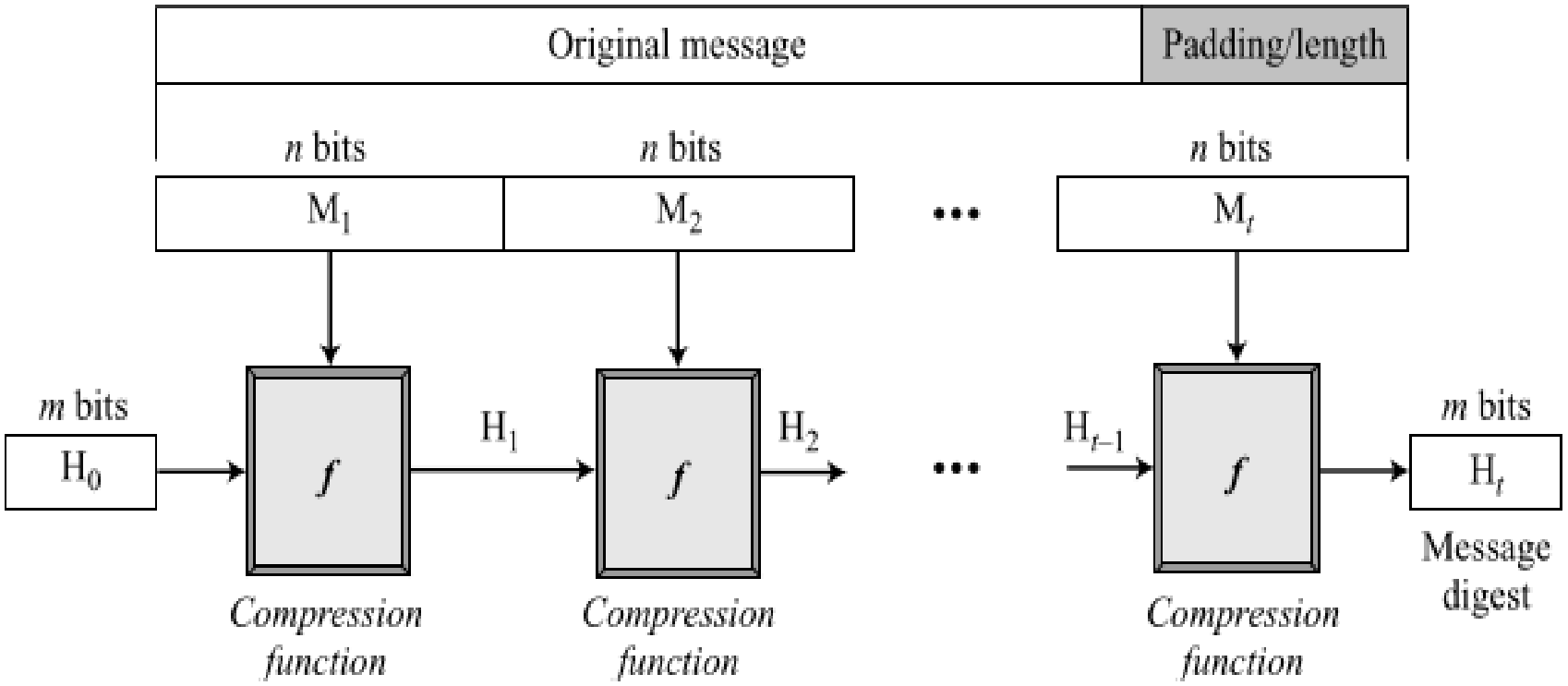
Cryptographic Hash function

- The kind of hash function needed for security applications is referred to as a cryptographic hash function.
- It has following properties.
- Hash function H must be one way.
- It means its easy to compute H(k) given k but its difficult to compute k given H(k).
- Hash function should have weak collision property. It means given a key x, it is hard to find another key y such that H(x)=H(y).

Cryptographic Hash Function

- All cryptographic hash functions need to create a fixed-size digest out of a variable-size message.
- However, we can create a hash function with fixed-size input and use it multiple times to create collision resistant digest.
- If size of input is n then hash function produces digest of size m
- Here, m<n.
- So, hash function is also known as compression function.

Generic Cryptographic Hash



- In generic hash function, message is divided into fixed size blocks n bits.
- If last block is not of n bits, the it is padded.

Specific Cryptographic Hash

- Some specific cryptographic hash functions are
- Message Digest MD, MD2, MD4, MD5
- Secure Hash Algorithm SHA, SHA1, SHA2 (SHA512, SHA256,SHA384)
- Each of these hash function differs in the digest size and block size.

Specific Cryptographic Hash

Table 11.3 Comparison of SHA Parameters

	SHA-1	SHA-224	SHA-256	SHA-384	SHA-512
Message Digest Size	160	224	256	384	512
Message Size	$< 2^{64}$	$< 2^{64}$	$< 2^{64}$	$< 2^{128}$	$< 2^{128}$
Block Size	512	512	512	1024	1024
Word Size	32	32	32	64	64
Number of Steps	80	64	64	80	80

Message Digest MD2

- Rivest first designed MD2 for 8-bit machines in 1989.
- Produces 128-bit hash using 128 bit blocks
- Described in RFC 1319
- MD2 along with RSA is used to generate digital certificates.
- Examples:
- MD2("The quick brown fox jumps over the lazy dog")
- = 03d85a0d629d2c442e987525319fc471
- But collisions were for MD2 were found soon

Message Digest MD4

- Rivest then developed MD4 for 32-bit machines in 1990.
- MD4 influenced a lot of cryptographic hash functions such as MD5, SHA-1.
- Same as MD2 collisions for MD4 were found soon.
- MD4 was designed to be fast which led to a lot of security risks.

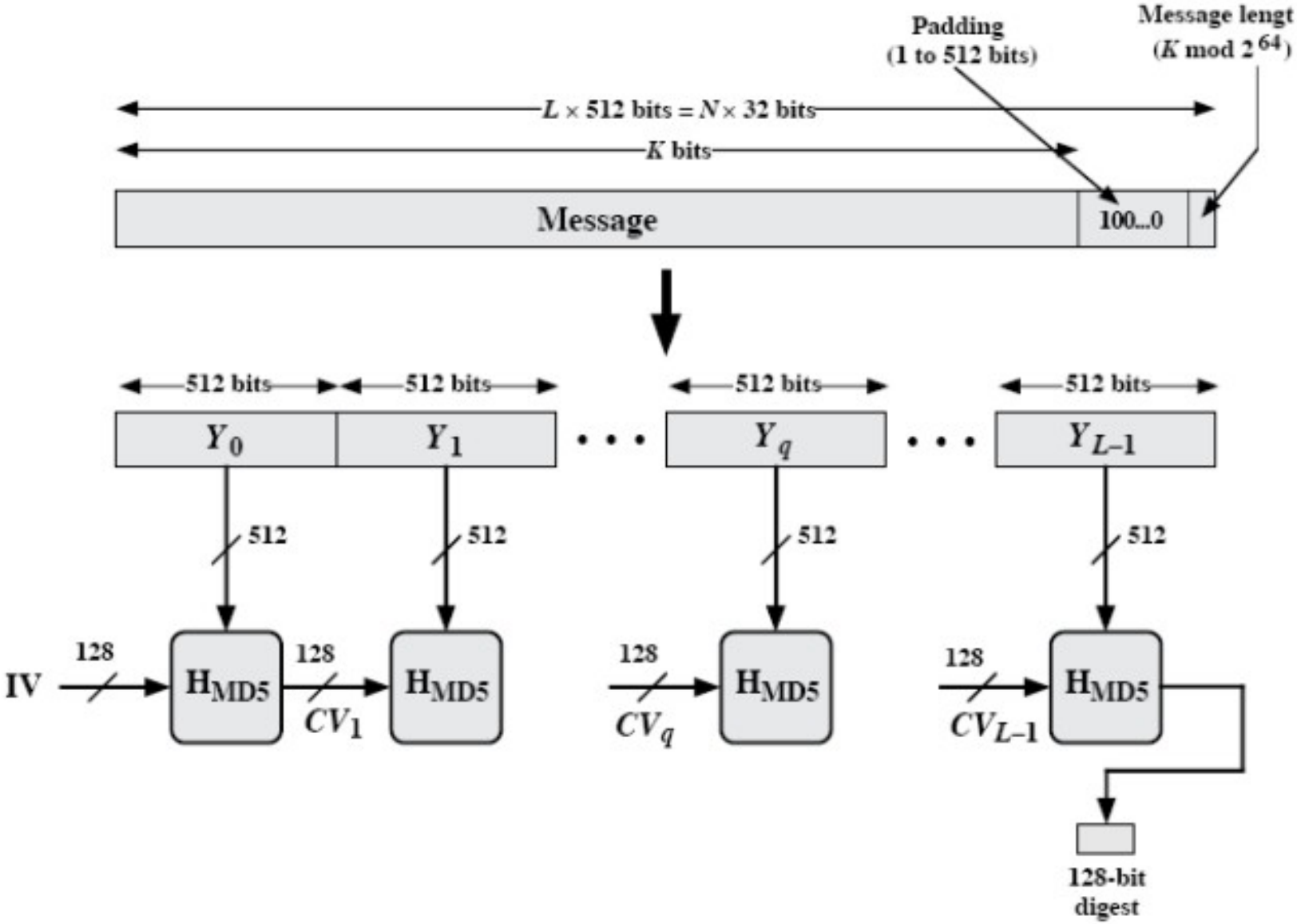
Message Digest MD5

- MD5 or ”message digest 5” algorithm was designed by professor Ronald Rivest at MIT.
- MD5 is a one way hashing function.
- MD5 creates a 128-bit message digest from the data input which is typically expressed in 32 digits hexadecimal number.
- MD5 hashes are unique for different inputs regardless of the size of the input.

MD5

- Step 1. Append the padding bits
- Step 2. Append the bit length
- Step 3. Initialization round constants
- Step 4. Initialize Buffers
- Step 5. Process the blocks
- Step 6: Generate Message Digest

MD5: Round Operation



MD5

- **Step 1: Pad the bits**
- For MD5 algorithm processes the input message in 512-bit blocks.
- So, following must be true for bit length l
- l≡ 448 mod 512.
- To make this true, we must pad some bits.
- The padding starts with bit 1 followed by the necessary number of 0-bits.
- Padded message is 64 bits less than 512 bits.

MD5

- **Step 1: Pad the bits**
- Let l=bit length of message, b be number of bits to be padded.
- Let l=200 bits
- Let b = number of bits to be padded
- Now l+b≡ 448 (mod 512)
- Or, b≡ 448-200 (mod 512)
- Or, b≡ 248 (mod 512)
- So, b=248

MD5

- **Step 1: Pad the bits**
- Let l=bit length of message, b be number of bits to be padded.
- IF l<448, b=448-l
- ELSE x=q*512+448
- Now put the value of q={1, 2, 3, ….} till x<l
- b=x-l
- For e.g., l=200 bits
- So, b=448-200=248

MD5

- **Step 1: Pad the bits**
- If l= 1000 bits
- Let b=number of bits to be padded
- Now, l+b≡ 448 (mod 512)
- Or b≡ 448-1000 (mod 512)
- Or b≡ -552 (mod 512)
- Or b≡ -40 (mod 512)
- Or b≡ 472 (mod 512)
- So, b=472

MD5

- **Step 1: Pad the bits**
- l=400 bits
- Let b=number of bits to be padded
- Now, l+b≡ 448 (mod 512)
- Or b≡ 448-400 (mod 512)
- Or b≡ 48 (mod 512)
- So, b=48

Message Digest MD5 Padding

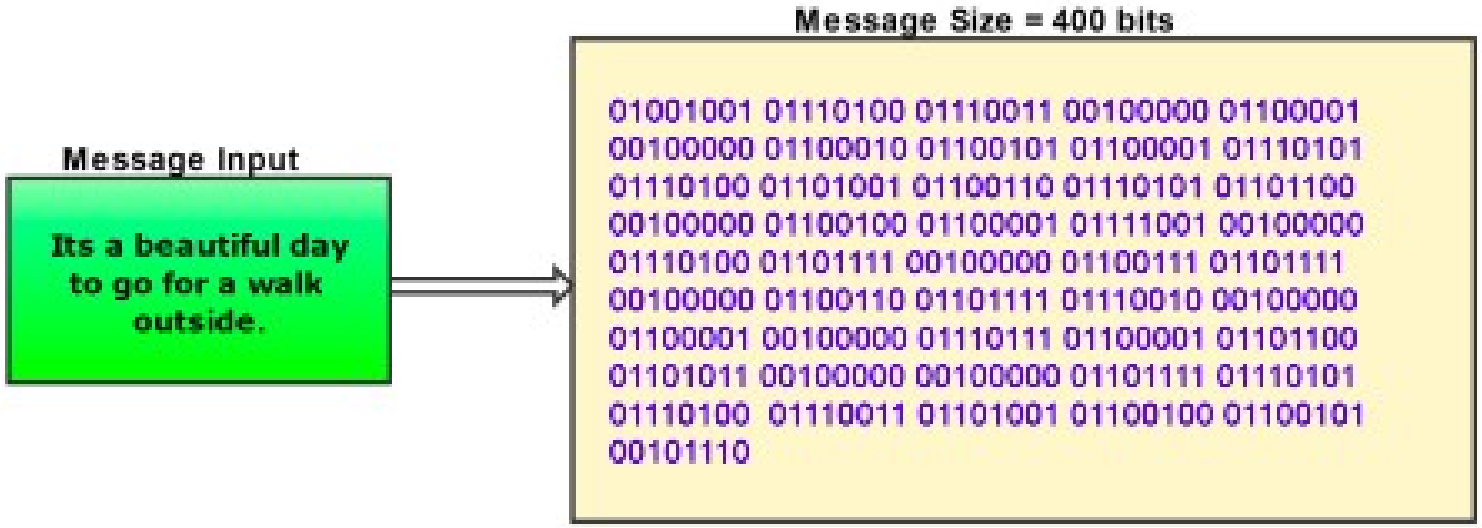


Figure 2: 400 bits original message

Message Digest MD5 Appending

- **Step 2: Append the bit length**
- Represent the bit length of message in 64 bit representation and then append it to the padded message.
- The appended message is exactly divisible by 512.
- For eg. if l=200
- The binary form of 200 is 11001000
- This is represented in 64 bit binary form and appended to padded message.
- 64 bit binary form is 0000000000000 …. 11001000

MD5

- **Step 3: Generate round constants.**
- The 64 round constants K[i] are computed using mathematical sin function:
- floor(abs(sin(i + 1)) × 2^{32})
- The angle is in radian.
- We denote the table as K[0.....63].
- For i=0,sin(0+1^{r})=0.841470985
- abs(sin(i + 1)) × 2^{32}=3614090361.10790656
- floor(abs(sin(i + 1)) × 2^{32})=3614090361
- In hexadiemal: (3614090361) = (D76AA477)

MD5: Process the blocks

K[0.. 3]	0xd76aa478, 0xe8c7b756, 0x242070db, 0xc1bdceee
K[4.. 7]	0xf57c0faf, 0x4787c62a, 0xa8304613, 0xfd469501
K[8..11]	0x698098d8, 0x8b44f7af, 0xffff5bb1, 0x895cd7be
K[12..15]	0x6b901122, 0xfd987193, 0xa679438e, 0x49b40821
K[16..19]	0xf61e2562, 0xc040b340, 0x265e5a51, 0xe9b6c7aa
K[20..23]	0xd62f105d, 0x02441453, 0xd8a1e681, 0xe7d3fbc8
K[24..27]	0x21e1cde6, 0xc33707d6, 0xf4d50d87, 0x455a14ed
K[28..31]	0xa9e3e905, 0xfcefa3f8, 0x676f02d9, 0x8d2a4c8a
K[32..35]	0xfffa3942, 0x8771f681, 0x6d9d6122, 0xfde5380c
K[36..39]	0xa4beea44, 0x4bdecfa9, 0xf6bb4b60, 0xbebfbc70
K[40..43]	0x289b7ec6, 0xeaa127fa, 0xd4ef3085, 0x04881d05
K[44..47]	0xd9d4d039, 0xe6db99e5, 0x1fa27cf8, 0xc4ac5665
K[48..51]	0xf4292244, 0x432aff97, 0xab9423a7, 0xfc93a039
K[52..55]	0x655b59c3, 0x8f0ccc92, 0xffeff47d, 0x85845dd1
K[56..59]	0x6fa87e4f, 0xfe2ce6e0, 0xa3014314, 0x4e0811a1
K[60..63]	0xf7537e82, 0xbd3af235, 0x2ad7d2bb, 0xeb86d391

MD5: Initialized MD Buffers

- **Step 4: Initialize MD Buffer**
- MD5 uses a four word buffer each 32-bits long. We denote them by A,B,C,D.
- These words are used in each step of each round.
- These are pre-initialized as:
- AA = 01 23 45 67
- BB = 89 ab cd ef
- CC = fe dc ba 98
- DD = 76 54 32 10

A=AA

B=BB

C=CC

D=DD

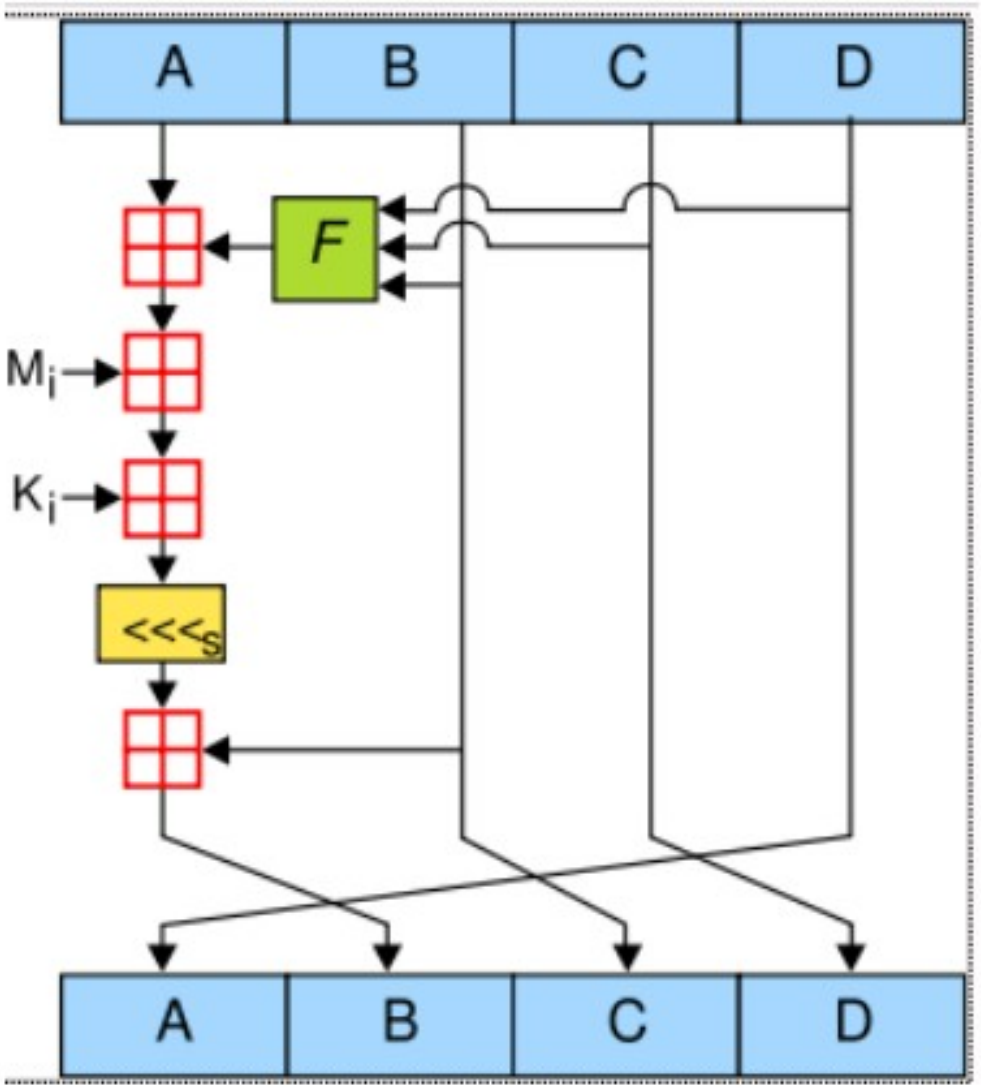
MD5

- **Step 5: Process the blocks**
- The message is divided into blocks of 512 bits.
- Each blocks are further divided into 16 sub-blocks or words M[0 1 … 15] of 32 bit.
- Round operation is performed for 4 rounds.
- Each round operation has specific functions as given below for round 1 to round 4.
- $F(B, C, D) = (B \wedge C) \vee (\neg B \wedge D)$
- $G(B, C, D) = (B \wedge D) \vee (C \wedge \neg D)$
- $H(B, C, D) = B \oplus C \oplus D$
- $I(B, C, D) = C \oplus (B \vee \neg D)$

MD5

- **Step 5: Process the blocks**
- Each round has 16 steps.
- Each step selects one of the message word and perform step operation.
- The selection of message word M[j] is based on following formula
- j=i for 0<=i<=15
- j=(5*i + 1) mod 16 for 16<=i<=31
- j=(3*i + 5) mod 16 for 32<=i<=47
- j=(7*i) mod 16 for 48<=i<=63

MD5: Step Operation



MD5

- **Step Operation**
- temp=D
- D=C
- C=B
- B = B + cls((A + F + K[i] + M[j]), s[i])
- A = temp
- Here addition is modulo 2^{32}
- The shift bits s[i] for different steps are given below

```
s[ 0..15] = {7,12,17,22,7,12,17,22,7,12,17,22,7,12,17,22}
s[16..31] = {5,9,14,20,5,9,14,20,5,9,14,20,5,9,14,20}
s[32..47] = {4,11,16,23,4,11,16,23,4,11,16,23,4,11,16,23 }
s[48..63] = {6,10,15,21,6,10,15,21,6,10,15,21,6,10,15,21 }
```

MD5

- **Step 6: Message Digest**
- Finally message digest is computed as given below
- AA=AA xor A
- BB=BB xor B
- CC=CC xor C
- DD=DD xor D
- Message digest is AA BB CC DD

SHA

- SHA was developed by the National Institute of Standards and Technology (NIST) in 1993 which is commonly known as SHA-0.
- When weaknesses were discovered in SHA-0, a revised version was issued as SHA-1 1995.
- SHA-1 uses block size of 512 and produces message digest of 160 bits.
- In 2002, NIST produced three new versions of SHA, with hash value lengths of 256, 384, and 512 bits, known as SHA-256, SHA-384, and SHA-512, respectively

SHA

- Collectively, these hash algorithms are known as SHA-2.
- In 2005, NIST announced the intention to phase out approval of SHA-1.
- A revised document was issued in 2008, which added a 224-bit version
- NIST move to a reliance on SHA-2 by 2010.

SHA1

- Step 1. Append the padding bits
- Step 2. Append the bit length
- Step 3. Initialization round constants
- Step 4. Initialize Buffers
- Step 5. Process the blocks
- Step 6: Generate Message Digest

SHA-1

- **steps: 1. Append padding bits**
- The message is padded so that its bit length after padding is congruent to 448 modulo 512
- l+b≡ 448 mod 512.
- Where, l =bit length of original message
- b=number of bits to be padded.
- That is, the length of the padded message is 64 bits less than an integer multiple of 512 bits.

SHA-1

- **steps: 1. Append padding bits**
- The padding consists of a single 1-bit followed by the necessary number of 0-bits.
- Let original message be
- 01100001 01100010 01100011 01100100 01100101
- Here length, l=40 bits
- l+b≡ 448 (mod 512).
- Or b≡ 448-l (mod 512).
- Or, b≡ 448-40 (mod 512)
- Or, b≡ 408 (mod 512)

SHA-1: padding bits

- Firstly 1 is padded and remaining 407 bits 0s are padded.
- Message after padding is
- 01100001 01100010 01100011 01100100 01100101 1
- In hex
- 61626364 65800000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

SHA-1

- **Step 2. Append the bit length**
- The length of original message as the block of 64 bits is then appended to the message.
- This block is treated as an unsigned 64-bit integer (most significant byte first) and contains the length of the original message (before padding).
- Here length of original message is 40 bits
- 40 in hex is 28

SHA-1

- **Step 2. Append the bit length**
- So, message after appending length is
- 61626364 65800000 00000000 00000000
  00000000 00000000 00000000 00000000
  00000000 00000000 00000000 00000000
  00000000 00000000 00000000 00000028

SHA-1: round keys

- **Step 3. Initialize round constant**
- Constant for each round is initialized as given below.
- K(t) = 0x5A827999 (0 <= t <= 19)
- K(t) = 0x6ED9EBA1 (20 <= t <= 39)
- K(t) = 0x8F1BBCDC (40 <= t <= 59)
- K(t) = 0xCA62C1D6 (60 <= t <= 79).

SHA-1

- **Step 4. Initialize MD buffer**
- A 160 bit buffer is used to hold intermediate values and final results of the Hash function represented as 5, 32 bit registers (h0,h1,h2,h3,h4) and (A, B, C, D, E) initialized as follows:
- h0 = 67452301
- h1 = EF CDAB89
- h2 = 98BADCF E
- h3 = 10325476
- h4 = C3D2E1F 0
- A = h0
- B = h1
- C = h2
- D = h3
- E = h4

SHA1

- **Step 5: Process the blocks**
- In this step following operations are performed.
- Creation of blocks
- Round operation
- Step operation

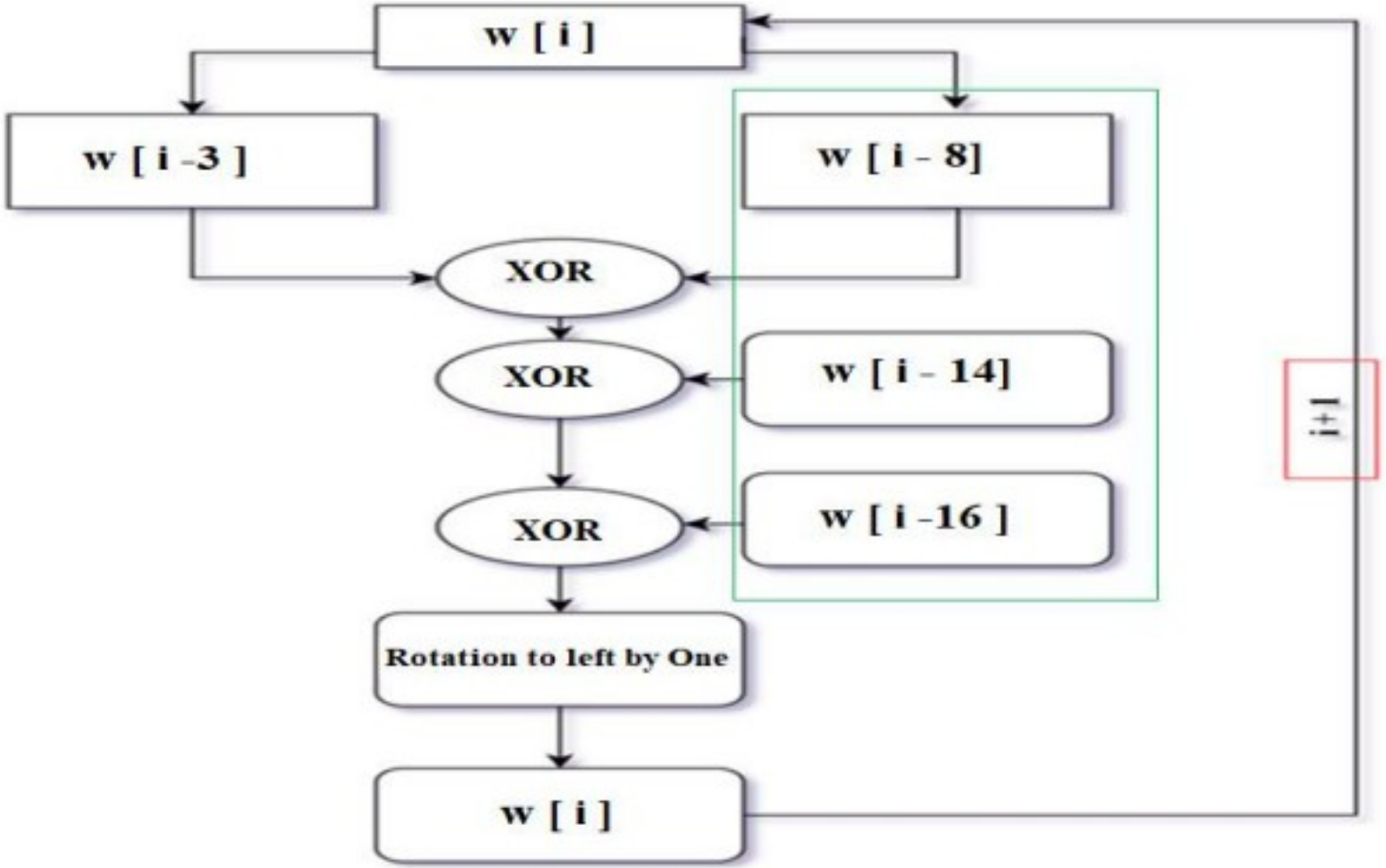
SHA-1

- **Step 5. Process the blocks**
- The message is divided into blocks of 512 bits.
- Total 80 words of 32 bits are generated from each block.
- Initial 16 words are directly generated from 512 bit block.
- These words are w_0, w_1, … w_{15}
- Other words from w_{16} to w_{79} are generated using following equation
- $w_t = w_{t-3}$ xor w_{t-8} xor w_{t-14} xor w_{t-16}
- Then left rotate the word by 1, $w_t = w_t$ Lrot 1

SHA-1

- **Step 5. Process the blocks**
- To generate words w_{16}, following equation is used
- $w_{16}=w_{13}\hat{}w_8\hat{}w_2\hat{}w_0$

SHA-1



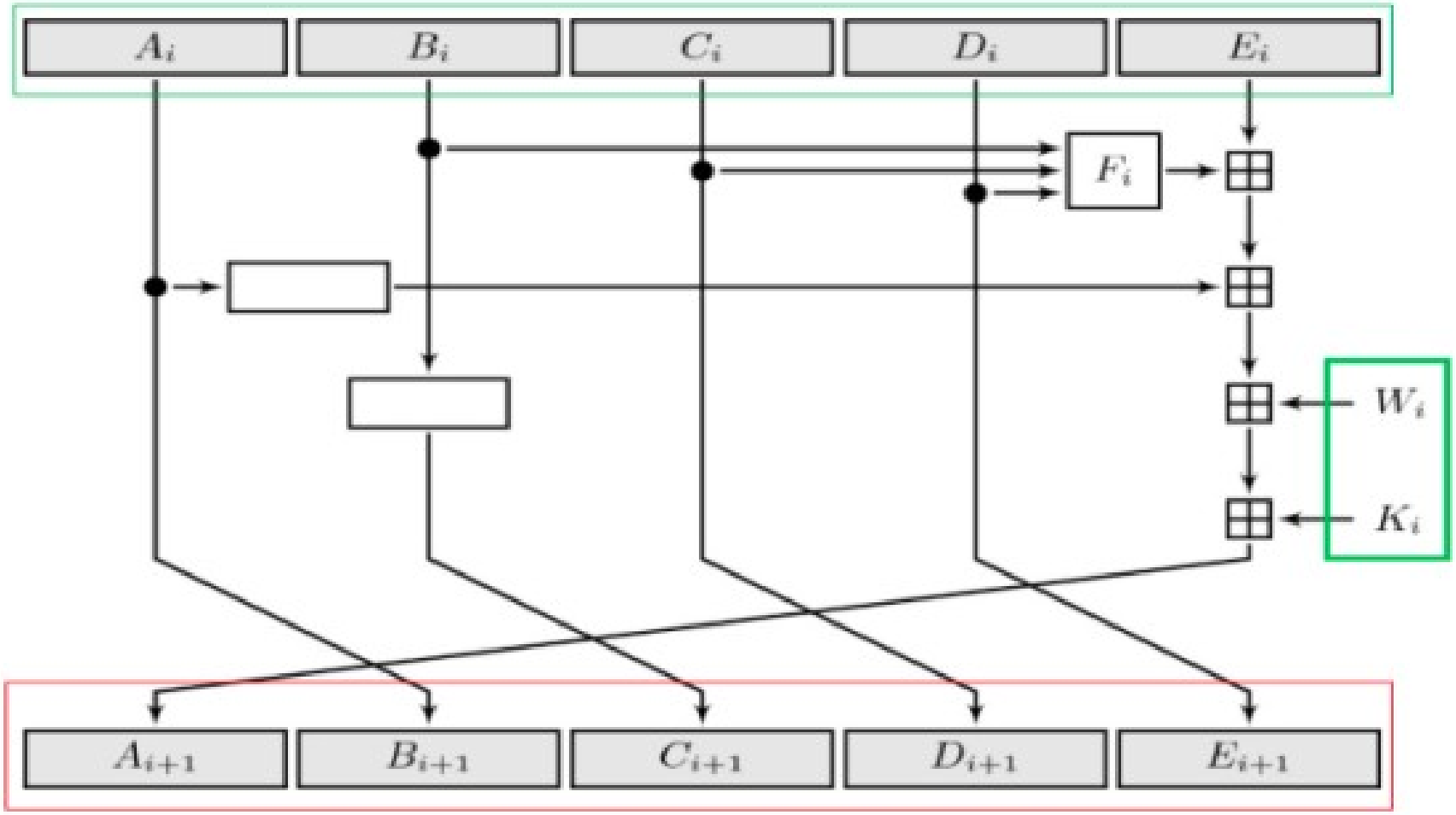
SHA-1

- **Step 5. Process the blocks**
- Each block undergoes round operation consisting of 20 steps.
- Each round has specific round function.
- Each round operation has similar structure but uses a different primitive logical function (f1, f2, f3 and f4).

SHA-1: Round functions

- Following four functions are used for four rounds each with 20 steps.
- f(t;B,C,D) = (B AND C) OR ((NOT B) AND D) (0 <= t <= 19)
- f(t;B,C,D) = B XOR C XOR D (20 <= t <= 39)
- f(t;B,C,D) = (B AND C) OR (B AND D) OR (C AND D) (40 <= t <= 59)
- f(t;B,C,D) = B XOR C XOR D (60 <= t <= 79).

SHA-1: step operation



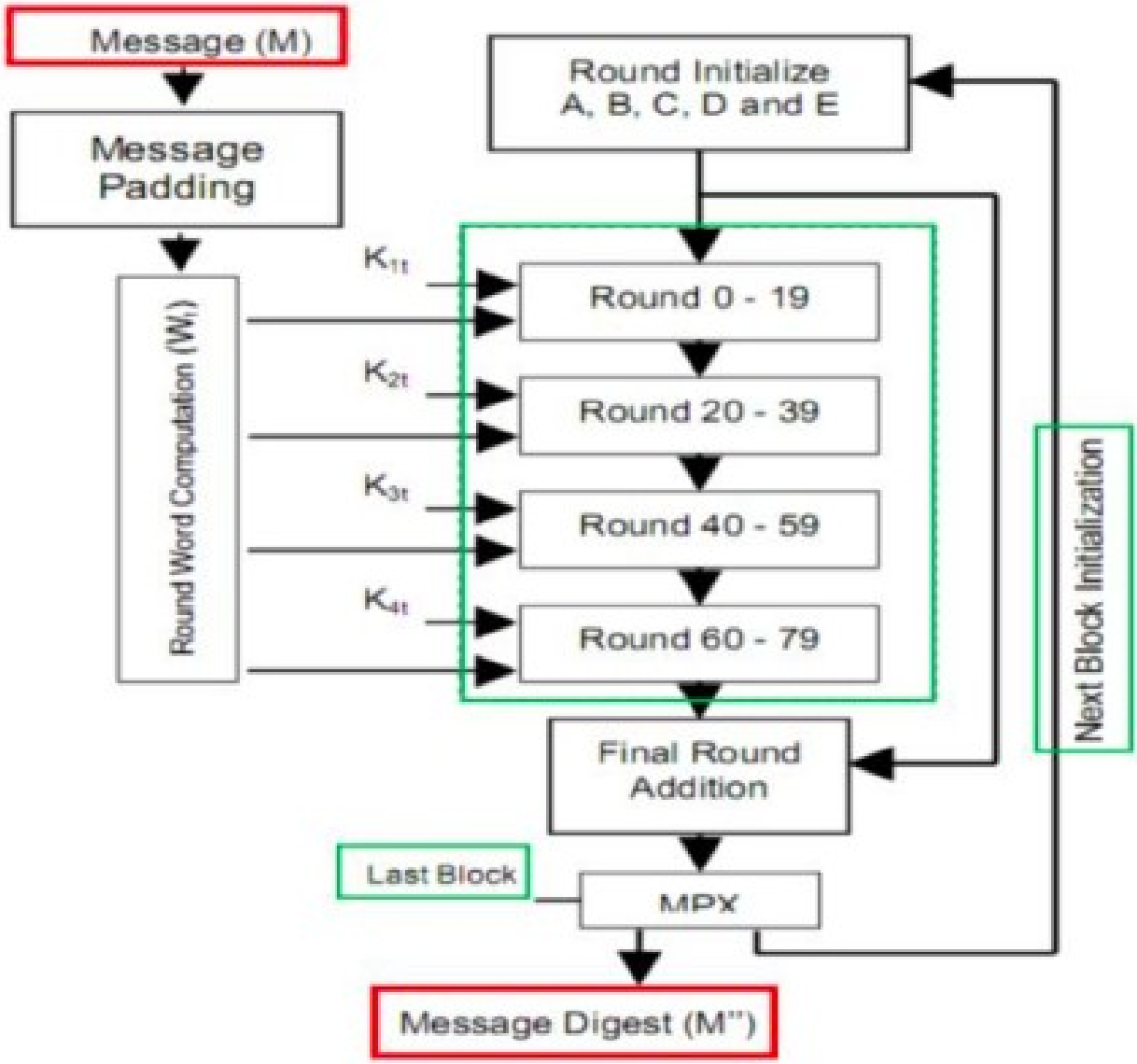
SHA-1: step operation

- Now following calculated
- Temp = A Lrot 5 + F + E + K_t + w_t
- Here addition is modulo 2^{32}
- Update the constant buffers
- E=D, D=C, C=B Lrot 30, B=A, A=temp
- This is done for round 1 to 4 and steps 0 to 79.

SHA-1: Message Digest

- **Step 6: Message Digest**
- After completing 4 rounds with 20 steps each, constants h0, h1, h2, h3, and h4 are updated as follows.
- h0=h0+A
- h1=h1+B
- h2=h2+C
- h3=h3+D
- h4=h4+E
- The addition operation is modulo 2^{32}.
- Finally message digest is h0h1h2h3h4

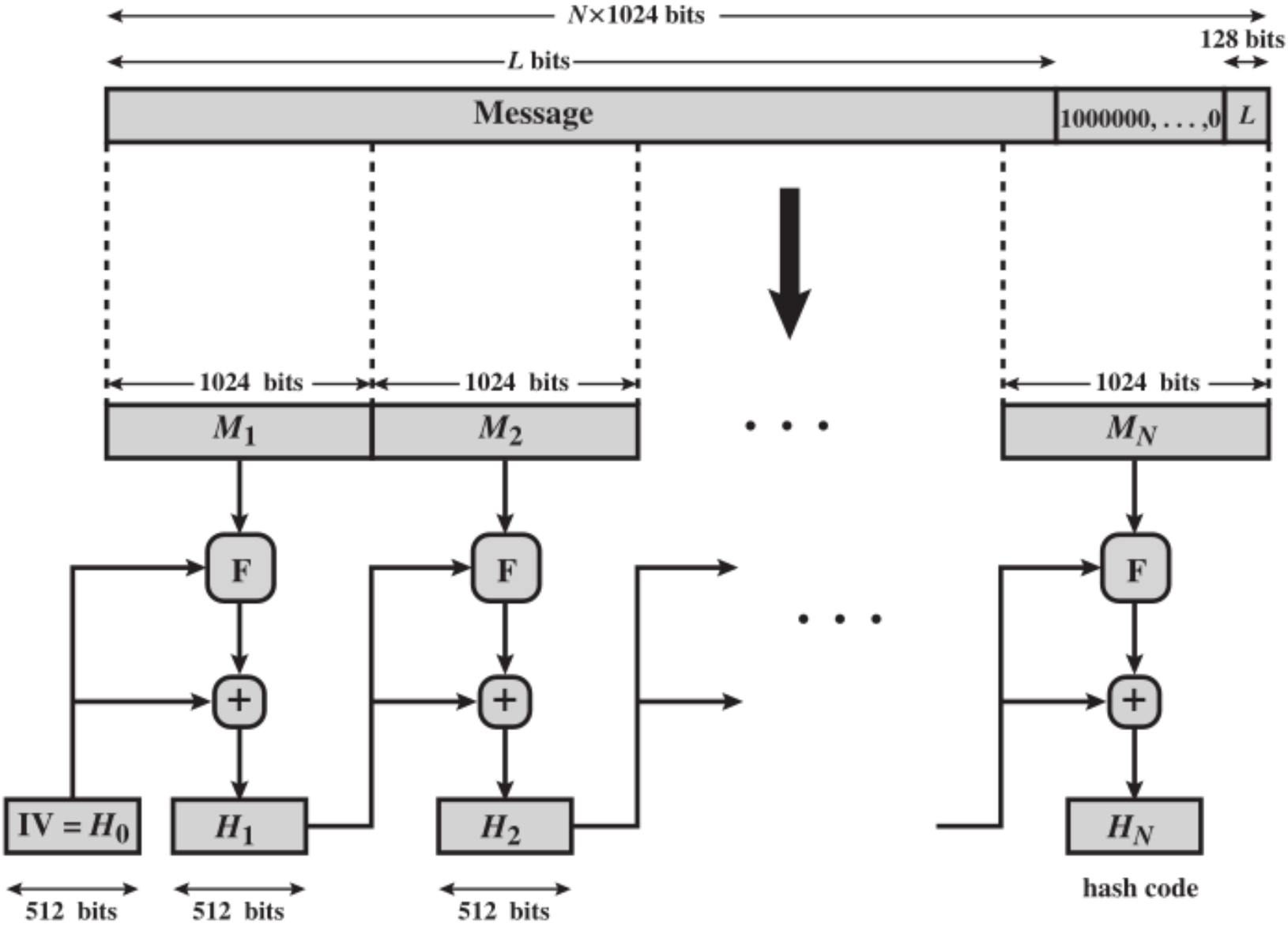
SHA-1


Message (M)
Message Padding
Round Word Computation (W_t)
Round Initialize A, B, C, D and E
K_{1t}
K_{2t}
K_{3t}
K_{4t}
Round 0 - 19
Round 20 - 39
Round 40 - 59
Round 60 - 79
Final Round Addition
Last Block
MPX
Message Digest (M'')
Next Block Initialization

SHA 512

- We provide a description of SHA-512.
- The other versions are quite similar.
- The algorithm takes as input a message with a maximum length of less than 2^{128} bits
- produces as output a 512-bit message digest.
- The input is processed in 1024-bit blocks.
- The Figure below depicts the overall processing of a message to produce a digest.

SHA 512


N×1024 bits
128 bits
L bits
Message
1000000, . . . ,0
L
1024 bits
1024 bits
1024 bits
M1
M2
MN
F
F
F
IV = H0
H1
H2
HN
512 bits
512 bits
512 bits
hash code

SHA 512 Steps

- **Step 1 Append padding bits**.
- The message is padded so that its length is congruent to 896 modulo 1024
- Padding is always added, even if the message is already of the desired length.
- Thus, the number of padding bits is in the range of 1 to 1024.
- The padding consists of a single 1 bit followed by the necessary number of 0 bits.

SHA 512 Steps

- **Step 1 Append padding bits**.
- Let the message be 01100001 01100010 01100011
- Message length=24 bits
- 24 is not congruent to 896 mod 1024
- Add 872 bits to messasge so that congruence holds.
- First padding bit is 1 and other 871 bits are 0s

SHA 512 Steps

- **Step 2 Append length**.
- A 128 bits is appended to the message to make the message the multiple of 1024 bits.
- This 128 bits represents the length of the original message before the padding.
- Expanded message is represented as the sequence of 1024-bit blocks M_1, M_2,……., M_N from N * 1024 bits of message

SHA 512 Steps

- **Step 3 Initialize hash buffer**.
- A 512-bit buffer is used to hold intermediate and final results of the hash function.
- The buffer can be represented as eight 64-bit registers (a, b, c, d, e, f, g, h).
- These registers are initialized to the following 64-bit integers (hexadecimal values):

SHA 512 Steps

- **Step 3 Initialize hash buffer**.

```
a = 6A09E667F3BCC908    e = 510E527FADE682D1
b = BB67AE8584CAA73B    f = 9B05688C2B3E6C1F
c = 3C6EF372FE94F82B    g = 1F83D9ABFB41BD6B
d = A54FF53A5F1D36F1    h = 5BE0CD19137E2179
```

- These values are stored in big-endian format, which is the most significant byte of a word in the low-address (leftmost) byte position.

SHA 512 Steps

- **Step 4 Process message in 1024-bit** (128-word) blocks.
- Blocks of 1024 bit and message digest of 512 bits are fed to module f.
- For initial block message digest is initialization vector of constants a,b,c,d,e,f,g,h.
- Each constant are of 64 bits.
- The output of module f and IV is then added modulo 2^{64} to produce the digest.

SHA 512 Steps

- **Module F Processing**
- Module F takes block of 1024 bits and generates 80 words each of 64 bits.
- We can get 16 words w_0 to w_{15} from 1024 bits.
- Other words are generated using following formula.

Application of Cryptographic Hash

- Message Authentication
- Digital Signature
- Password Storage
- Pseudo Random number generator

Password security

- Hash functions are commonly used to create a one-way password file.
- It helps to hash password and is stored by an operating system rather than the password itself.
- The hash of password does not help the attacker to generate the password because the hash function is one way.

Random Number Generation

- A cryptographic hash function can be used to construct a pseudo random function (PRF) or a pseudo random number generator (PRNG).
- A common application for a hash-based PRF is for the generation of symmetric keys.

Message Authentication

- Message authentication is a mechanism or service used to verify the source.
- It assures that the data is from alleged source.
- Message integrity assures that data received are exactly as sent (i.e., contain no modification, insertion, deletion, or replay).
- When a hash function is used to provide message authentication, the hash function value is often referred to as a **message digest**.

Message Authentication

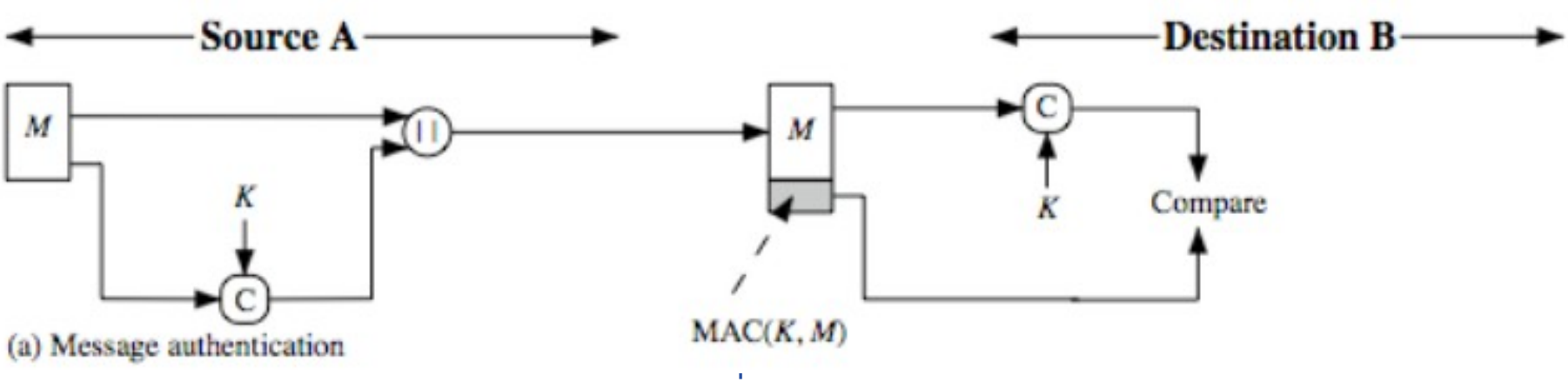
- Message authentication can be done in following ways.
- Symmetric encryption and decryption.
- Public encryption and decryption
- Digital signature
- Hash function

Message Authentication with Symmetric Encryption

- The sender encrypts the message with the shared key.
- Encrypted message is known as Message Authentication Code (MAC) or Authenticator.
- The sender sends message appending MAC.
- It is necessary to send the message also so that original message and decrypted message can be compared.
- Otherwise, some message cannot be intelligible after decryption and cannot be authenticated.

Message Authentication with Symmetric Encryption

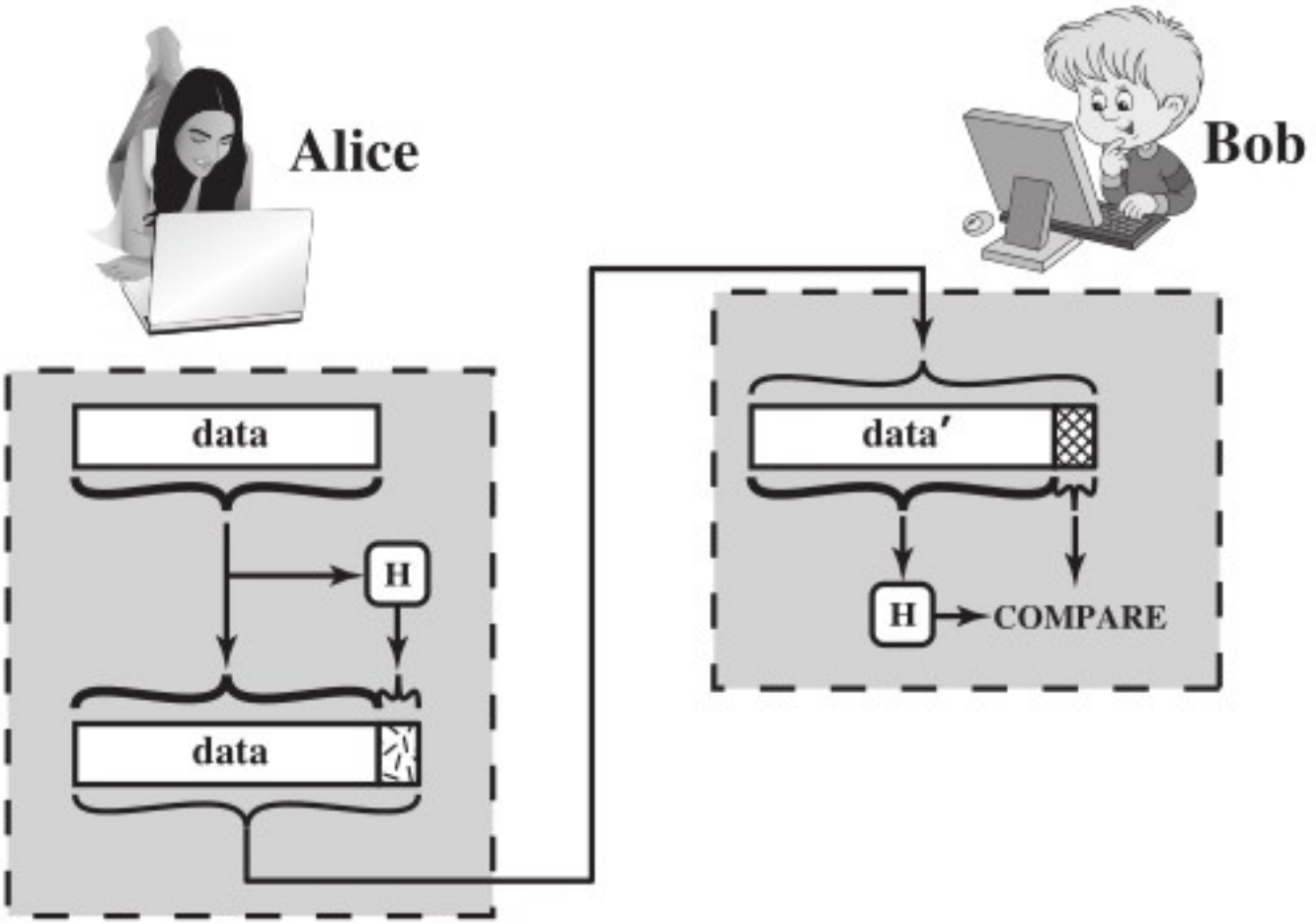
- Receiver receives the message and decrypts the MAC.
- If original message is same to decrypted MAC, then it proves integrity of message.
- It also proves source of message as the secret key is only shared between sender and receiver.



Message Authentication with Hash

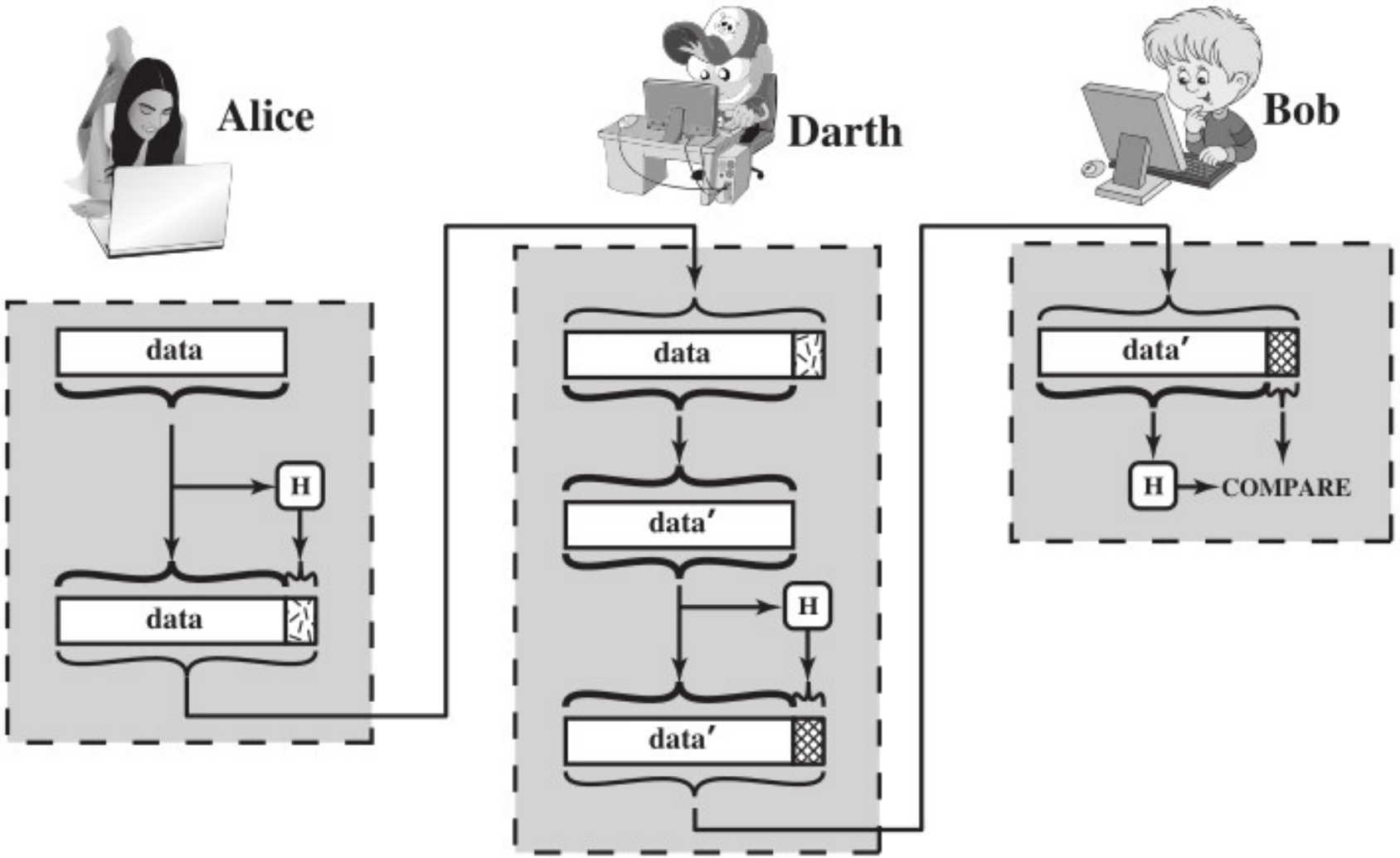
- It will be difficult to generate MAC using encryption for long message.
- Because size of ciphertext will be as long as the size of plaintext.
- Such a long ciphertext will be difficult to transmit also.
- So, in such case we can use cryptographic hash functions such as MD4, SHA-1.
- Has function compress the plaintext.
- However, it can face man in the middle attack.

Process of Message Authentication



(a) Use of hash function to check data integrity

Man in Middle Attack

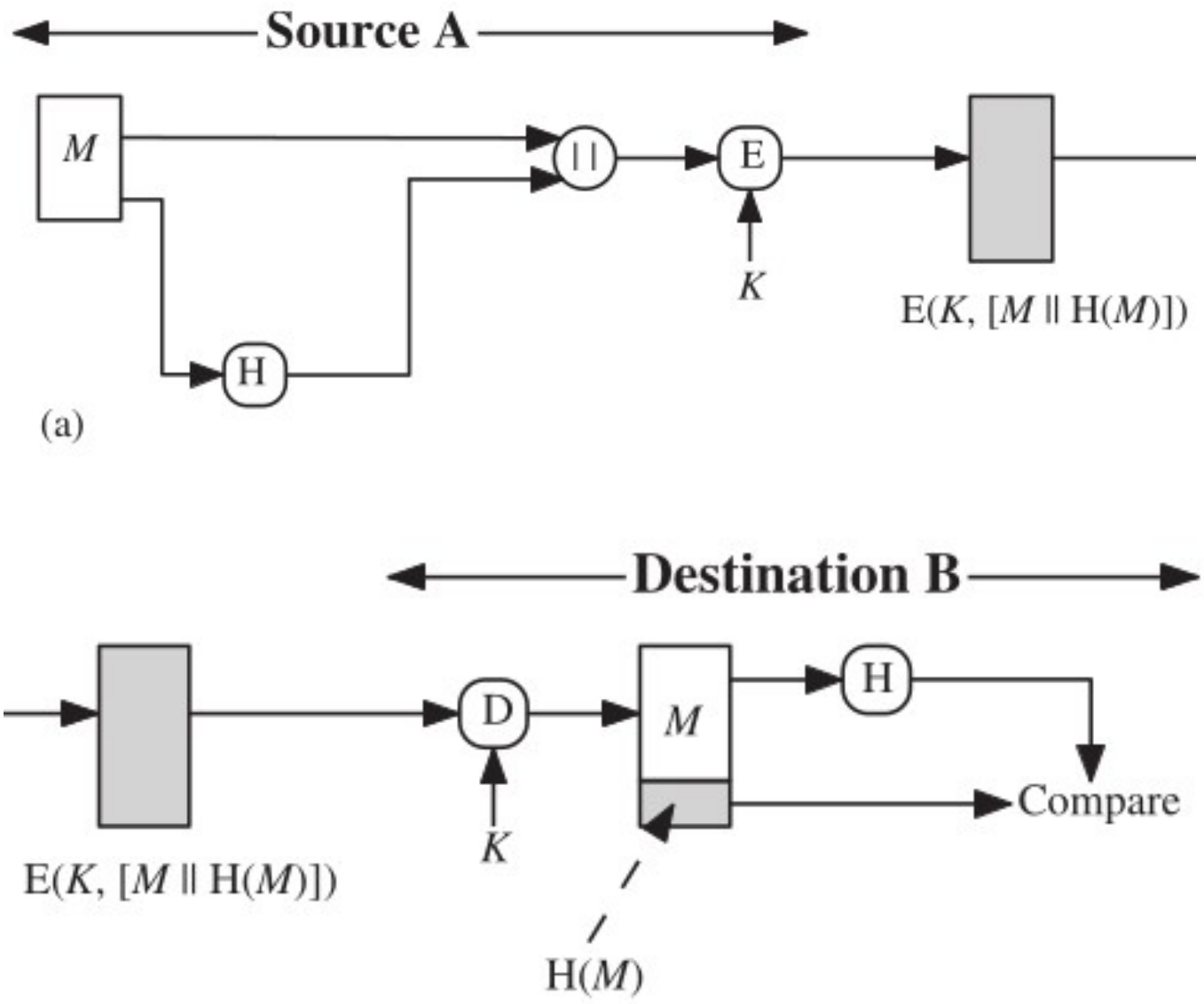


(b) Man-in-the-middle attack

Encrypting Hash value

- One of the solution to man in the middle attack can be encrypting hash function itself.
- This helps to transmit hash function in a secure fashion.
- Now it will be difficult for adversary to alters or replaces the message.
- It is not feasible for adversary to alter the hash value also.

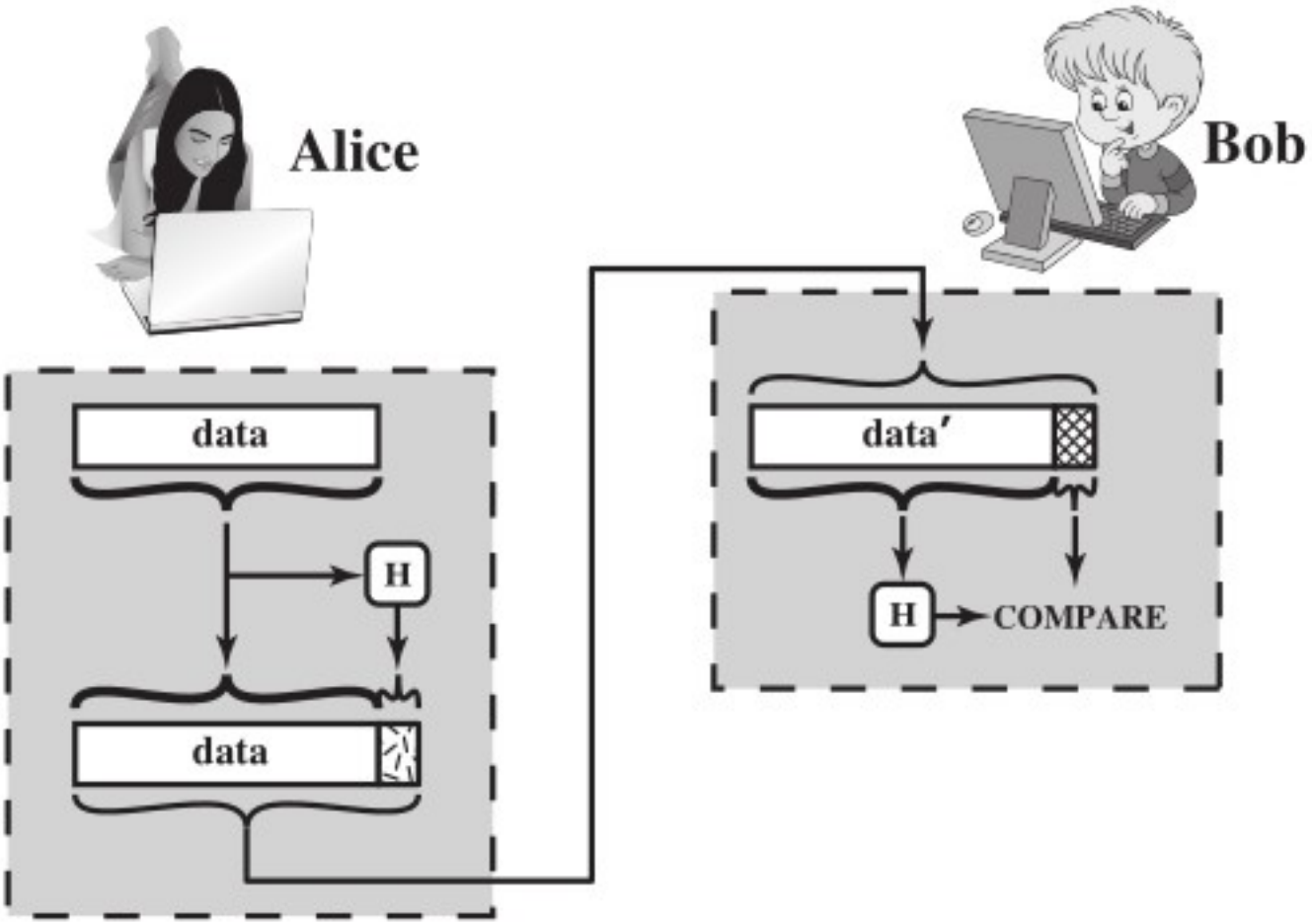
Encrypting Hash value



Problem with message authentication

- Message authentication using encryption or hash function can protect two communicating parties from any third party.
- However, it does not protect the two parties against each other.
- Several forms of dispute between the two are possible.
- For example, suppose that Alice sends an authenticated message to Bob using one of the schemes described earlier.

Problem with message authentication



(a) Use of hash function to check data integrity

Problem with message authentication

- Bob may forge a different message and claim that it came from Alice.
- Bob would simply have to create a message and append an authentication code using the key that Alice and Bob share.
- Similarly, Alice can deny sending the message saying that Bob itself can generate message.

Problem with message authentication

- Both scenarios are of legitimate concern.
- In situations where there is not complete trust between sender and receiver, something more than authentication is needed.
- The most attractive solution to this problem is the digital signature.
- The digital signature is analogous to the handwritten signature.

Digital Signatures

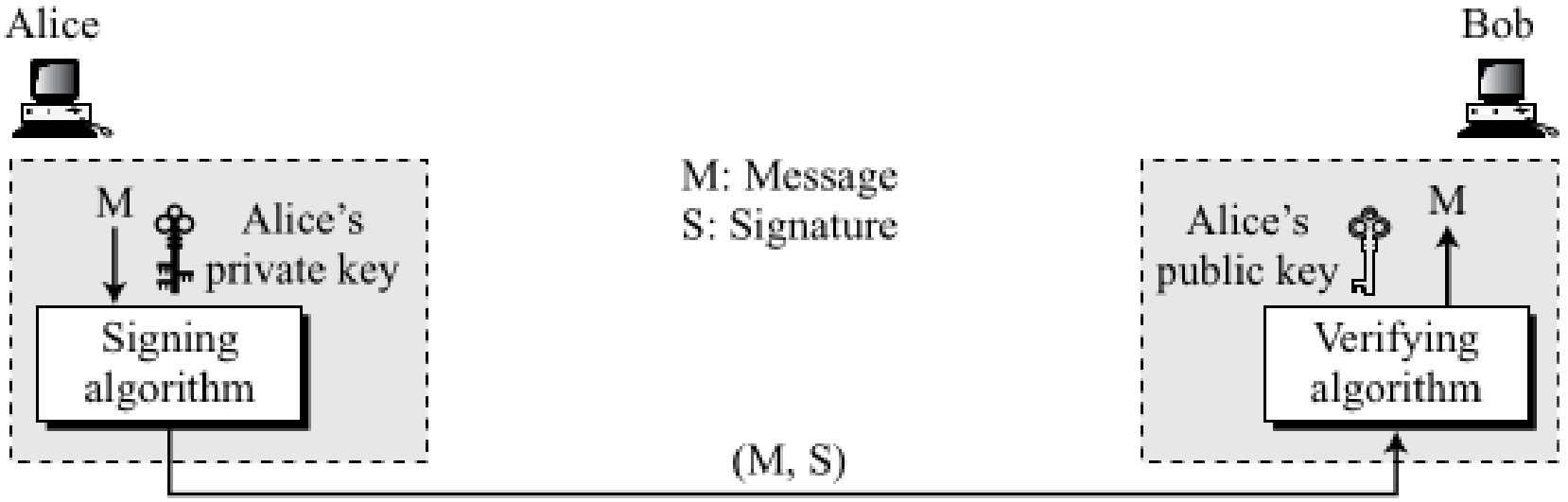
- Digital signature is used to sign the digital document or message.
- It ensures the services like integrity, authentication and non-repudiation.
- To sign the message with digital signature, the sender most have its pair of keys signing key and verifying key.
- Signing key can be user’s private key and verifying key is public key.

Digital Signatures

- The signer uses signing algorithm to sign the message.
- Receiver uses verifying algorithm to verify the message.
- Signing and verifying algorithm can be any public key cryptography such as RSA, Elgamal
- In this case, an attacker who wishes to alter the message would need to know the user's private key.

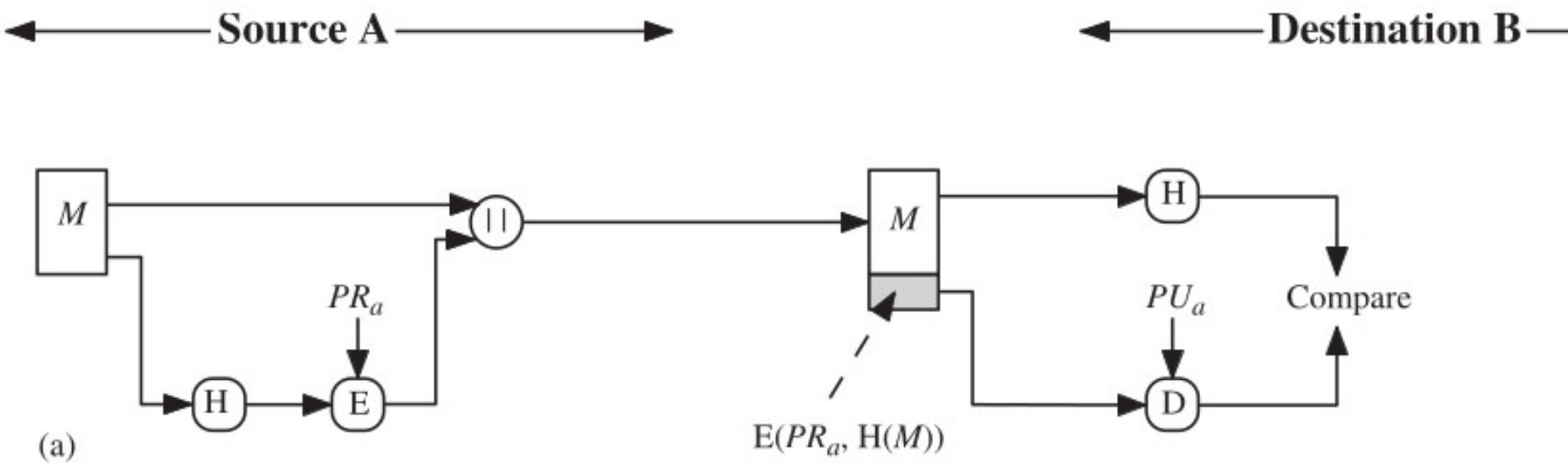
Digital Signatures

- Following process achieves message and user authentication.
- It also provides non-repudiation also.



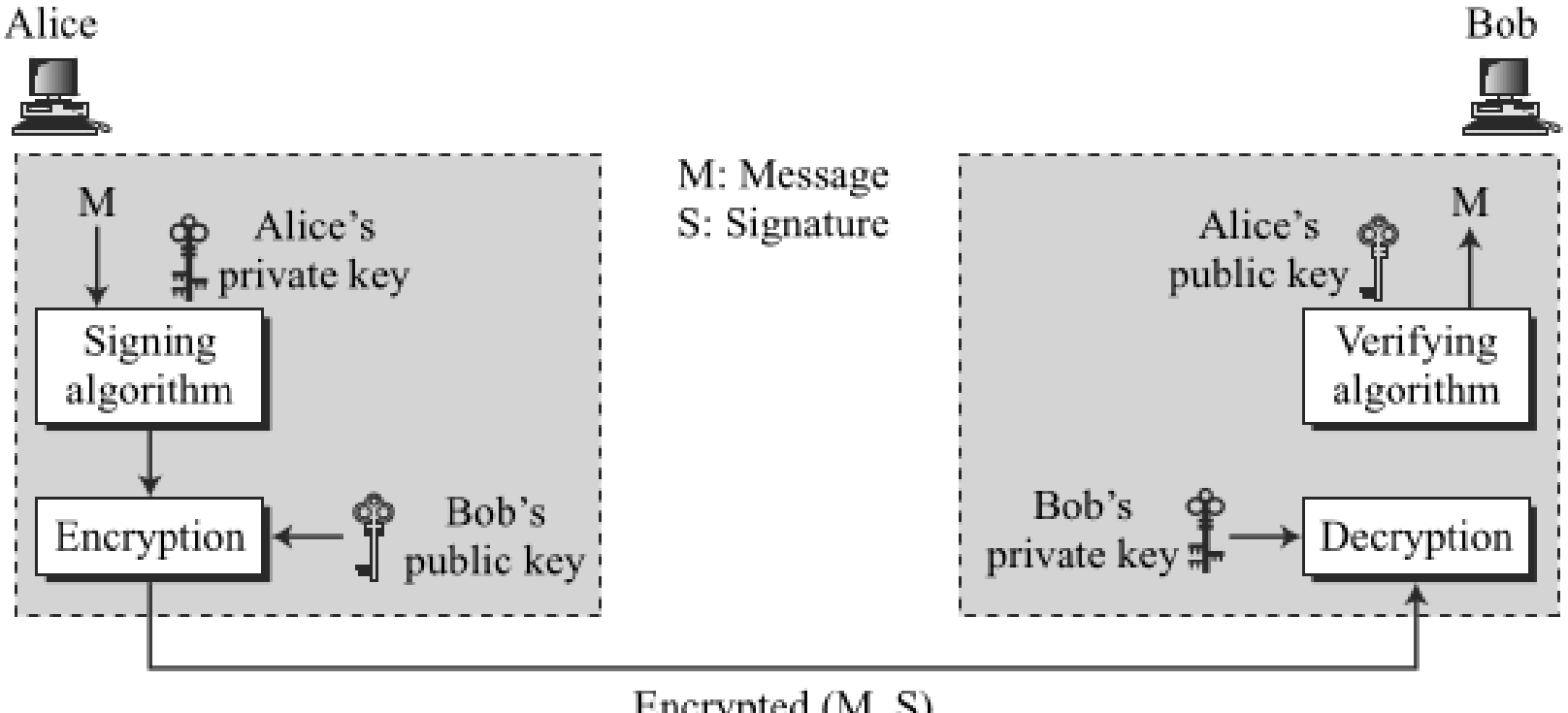
Digital Signatures

- If message long then it can be shortened by using some some cryptographic functions such as SHA-1.
- Then it can be signed using digital signature.



Digital Signatures

- In earlier version of digital signature, no confidentiality.
- To provide **confidentiality** if we encrypt the message and signature with public key.



Digital Signatures

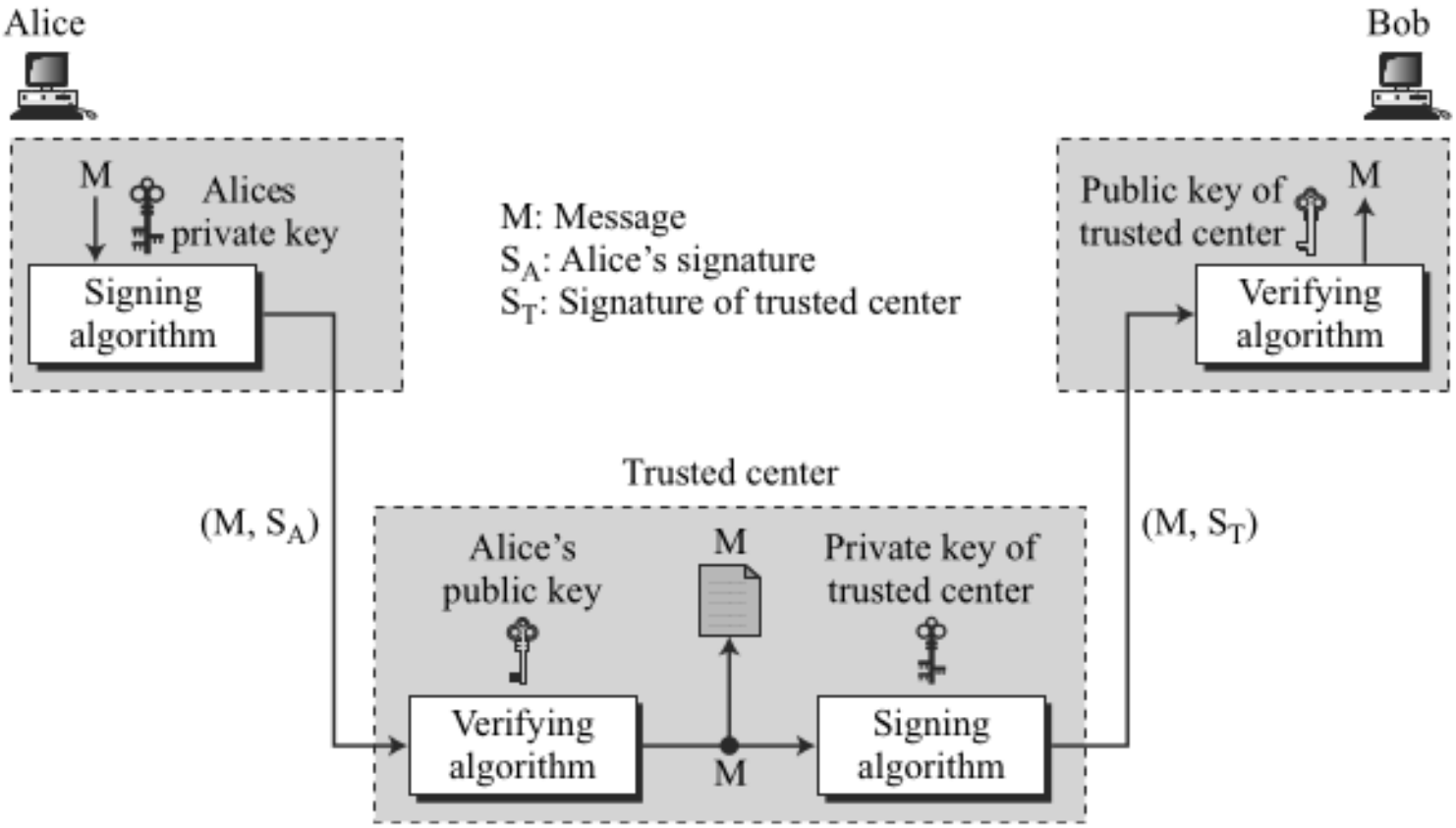
- **Non-repudiation**
- If digital signatures are generated by user itself, the they can deny later that it is not my signature.
- For e.g. Alice sends message to Bob saying transfer $10,000 from her account to Ted's account,
- With the scheme we have presented so far, Bob might have a problem.
- Bob must keep the signature on file and later use Alice's public key to create the original message.

Digital Signatures

- **Non-repudiation**
- This is not feasible because Alice may have changed her private or public key during this time; she may also claim that the file containing the signature is not authentic.
- One solution is trusted center.
- Trusted center generates private and public key for each user.
- Trusted center verifies the signature.
- Trusted center helps for authenticated communication.

Digital Signatures

- Message can be signed with the help of trusted center.



Digital Signature

- It must have the following properties:
- It must verify the author and the date and time of the signature.
- It must authenticate the contents at the time of the signature.
- It must be verifiable by third parties, to resolve disputes.

Digital Signature

- Thus, the digital signature function includes the authentication function.
- On the basis of these properties, we can formulate the following requirements for a digital signature:
- The signature must be a bit pattern that depends on the message being signed.
- The signature must use some information unique to the sender, to prevent both forgery and denial.
- It must be relatively easy to produce the digital signature.

Digital Signature

- It must be relatively easy to recognise and verify the digital signature.
- It must be computationally infeasible to forge a digital signature, either by constructing a new message for an existing digital signature or by constructing a fraudulent digital signature for a given message.
- It must be practical to retain a copy of the digital signature in storage.

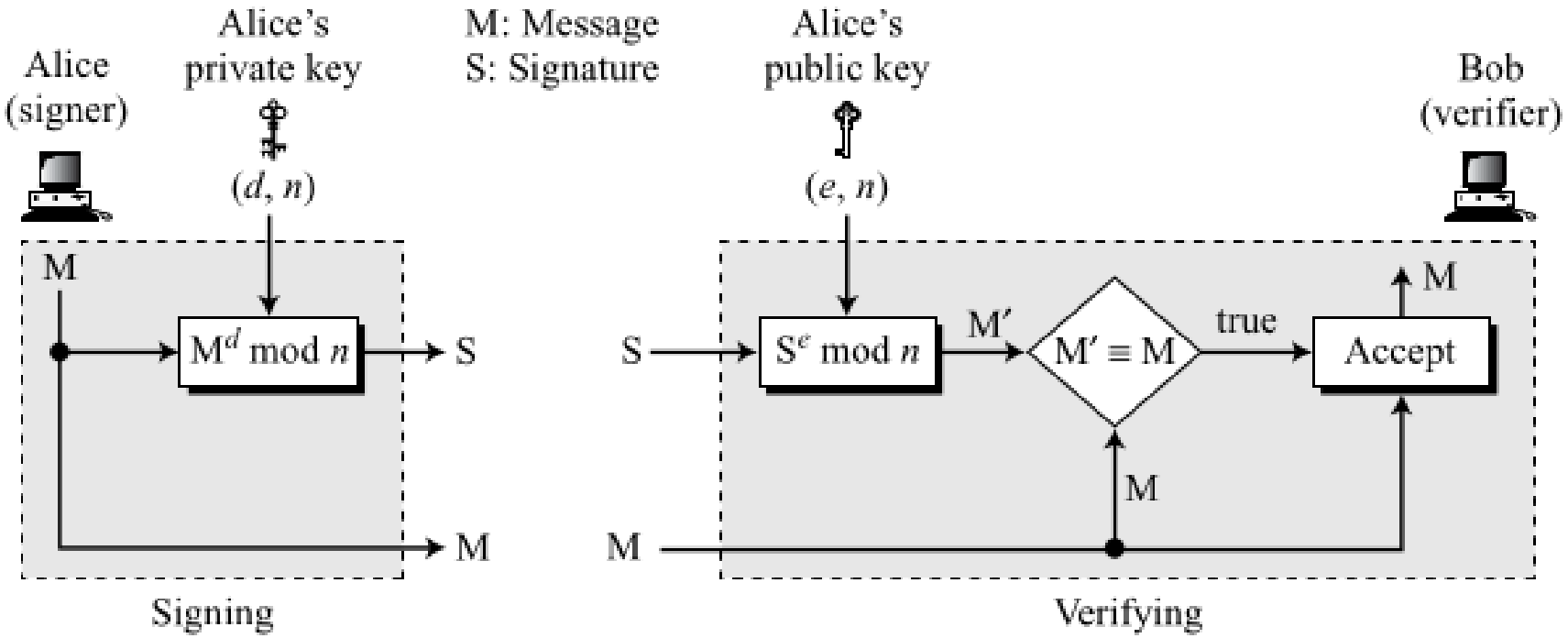
Digital Signature Schemes

- The concept of digital signature is implemented using different public cryptography algorithms such as RSA, Elgamal etc
- Those implementations are digial signature schemes.
- We will mainly focus on RSA digial signature.

RSA Digital Signature Schemes

- RSA digital signature scheme changes the roles of the private and public keys.
- First, the private and public keys of the sender, not the receiver, are used.
- Second, the sender uses her own private key to sign the document;
- The receiver uses the sender's public key to verify it.

RSA Digital Signature Schemes



RSA Digital Signature Schemes

- It includes 3 steps
- Key generation
- Signing the message
- Verifying the message

RSA Digital Signature Key Generation

- Key generation in the RSA digital signature scheme is exactly the same as key generation in the RSA cryptosystem.
- Alice chooses two primes p and q
- calculates $n = p \times q$.
- Alice calculates $\varphi(n) = (p - 1)(q - 1)$.
- She then chooses e,
- calculates d, such that $e \times d = 1 \bmod \varphi(n)$.
- The key (d,n) is private and
- (e,n) is public

RSA Digital Signature: Signing

- Alice creates a signature out of the message using her private exponent,
- $S = m^d \bmod n$
- Then sends the message and the signature to Bob.

RSA Digital Signature: Verifying

- Bob receives M and S.
- Bob applies Alice's public key to create a copy of the message
- $M' = S^e \bmod n$.
- Bob compares the value of M′ with the value of M.
- If the two values are congruent, Bob accepts the message.
- This provides authentication, integrity and non-repudiation
- This is because only Alice has private key.

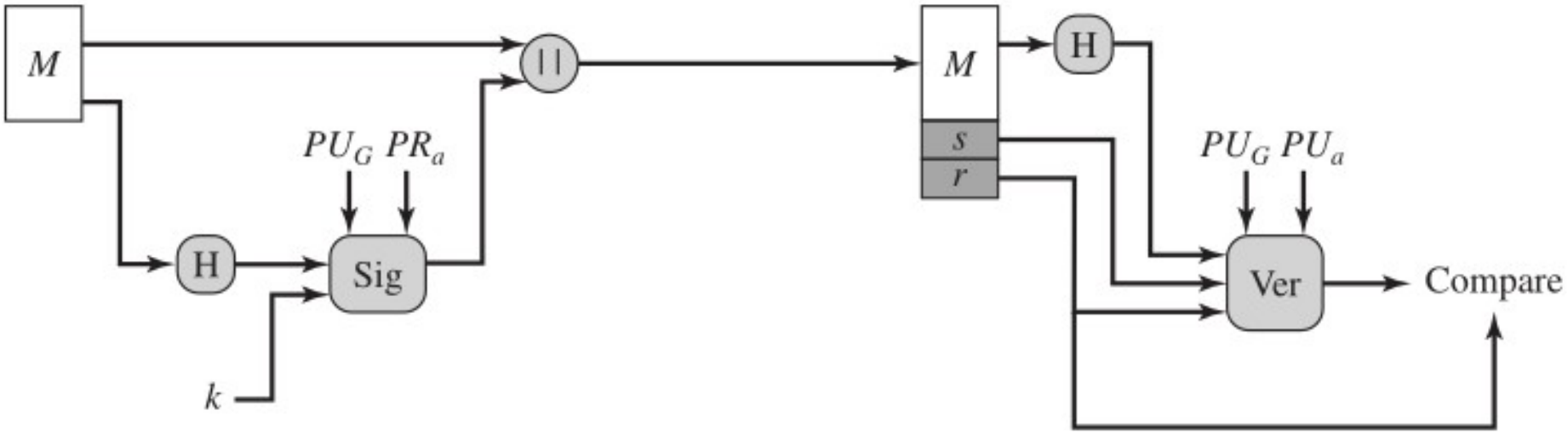
Digital Signature Standard

- The National Institute of Standards and Technology (NIST) has published Digital Signature Standard.
- It is based on Elgamal algorithm
- The DSS was originally proposed in 1991 and revised in 1993 in response to public feedback.
- There was a further minor revision in 1996.
- DSS includes Digital Signature Algorithm based upon Elgammal and hash algorithm SHA-1.

Digital Signature Standard

- DSA is used to sign the message.
- DSA needs small fixed size message to sign it.
- So, to convert arbitrary sized message to fixed size message, SHA-1 operates on the original message.
- SHA-1 produces 160 bit message digest.
- Then, message digest is signed with DSA.

Digital Signature Standard



(b) DSA approach

Digital Signature Standard

- The signature function also depends on the sender's private key (PRa) and a set of parameters known to a group of communicating principals.
- We can consider this set to constitute a global public key (PUG).
- The result is a signature consisting of two components, labeled s and r.
- At the receiving end, the hash code of the incoming message is generated.

Digital Signature Standard

- This plus the signature is input to a verification function.
- The verification function also depends on the global public key as well as the sender's public key (PUa), which is paired with the sender's private key.
- The output of the verification function is a value that is equal to the signature component r if the signature is valid.
- The signature function is such that only the sender, with knowledge of the private key, could have produced the valid signature.

DSA: Global Public Key Generation

- So, DSA also has 3 steps.
- Key generation
- Signing message
- Verifying message

DSA: Global Public Key Generation

- A prime number p is selected such that $2^{L-1}<p<2^{L}$ where L is multiple of 64 and 512<=L< 1024 bits
- Bit size of p is L.
- Select a 160-bit prime number q from $2^{159}<q<2^{160}$ such that q|(p-1).
- Compute $g = h^{(p-1)/q} \bmod p$, where h is an integer between 14 and (p − 1) with the restriction that g must be greater than 1.
- The g, p, q are global keys shared between communicant.

DSA: public and private key

User's Private Key

x random or pseudorandom integer with $0 < x < q$

User's Public Key

$y = g^x \bmod p$

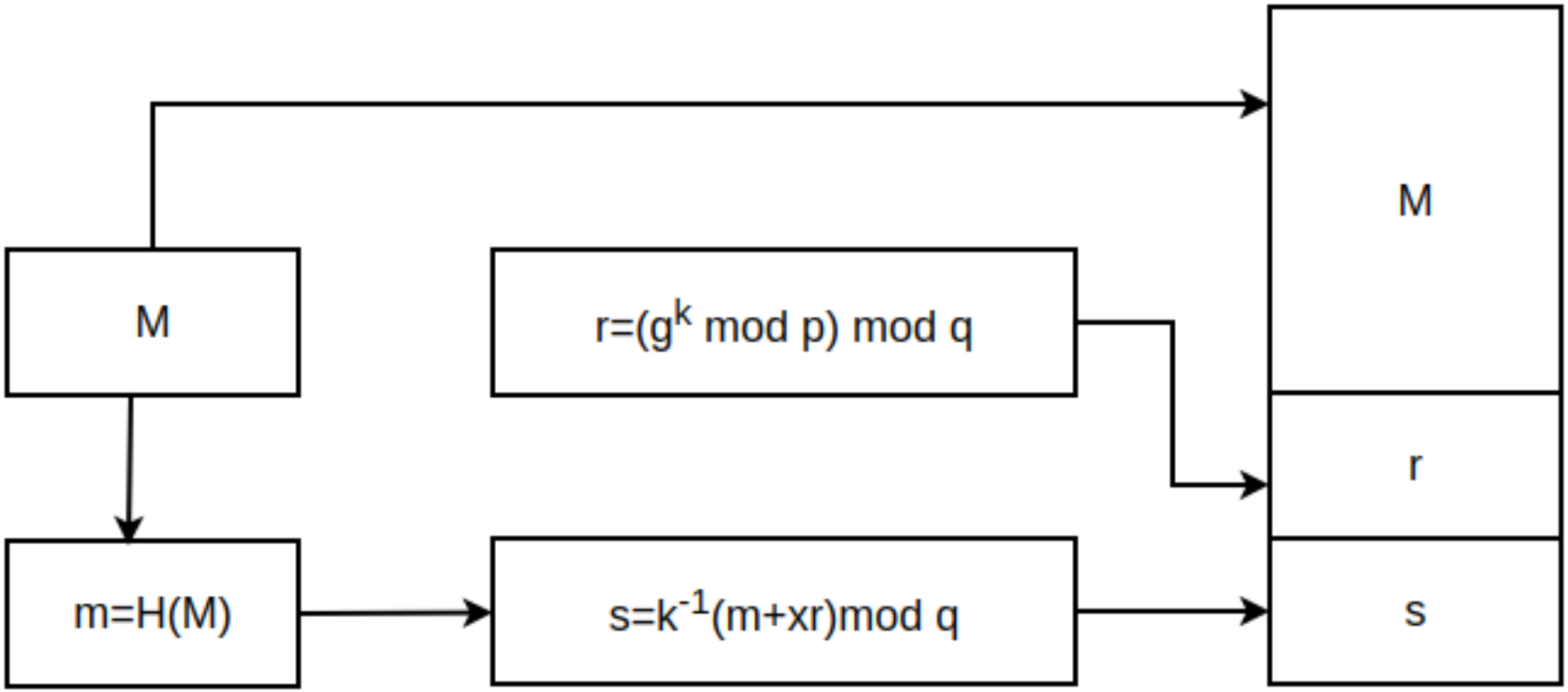
User's Per-Message Secret Number

k random or pseudorandom integer with $0 < k < q$

DSA: Signing

- The m be message of arbitrary big sized message.
- The m is hashed with SHA-1 hash H(M) for message M
- Compute followings
- m=H(M)
- $r= (g^k \bmod p) \bmod q$
- $s=k^{-1}(m+xr) \bmod q$

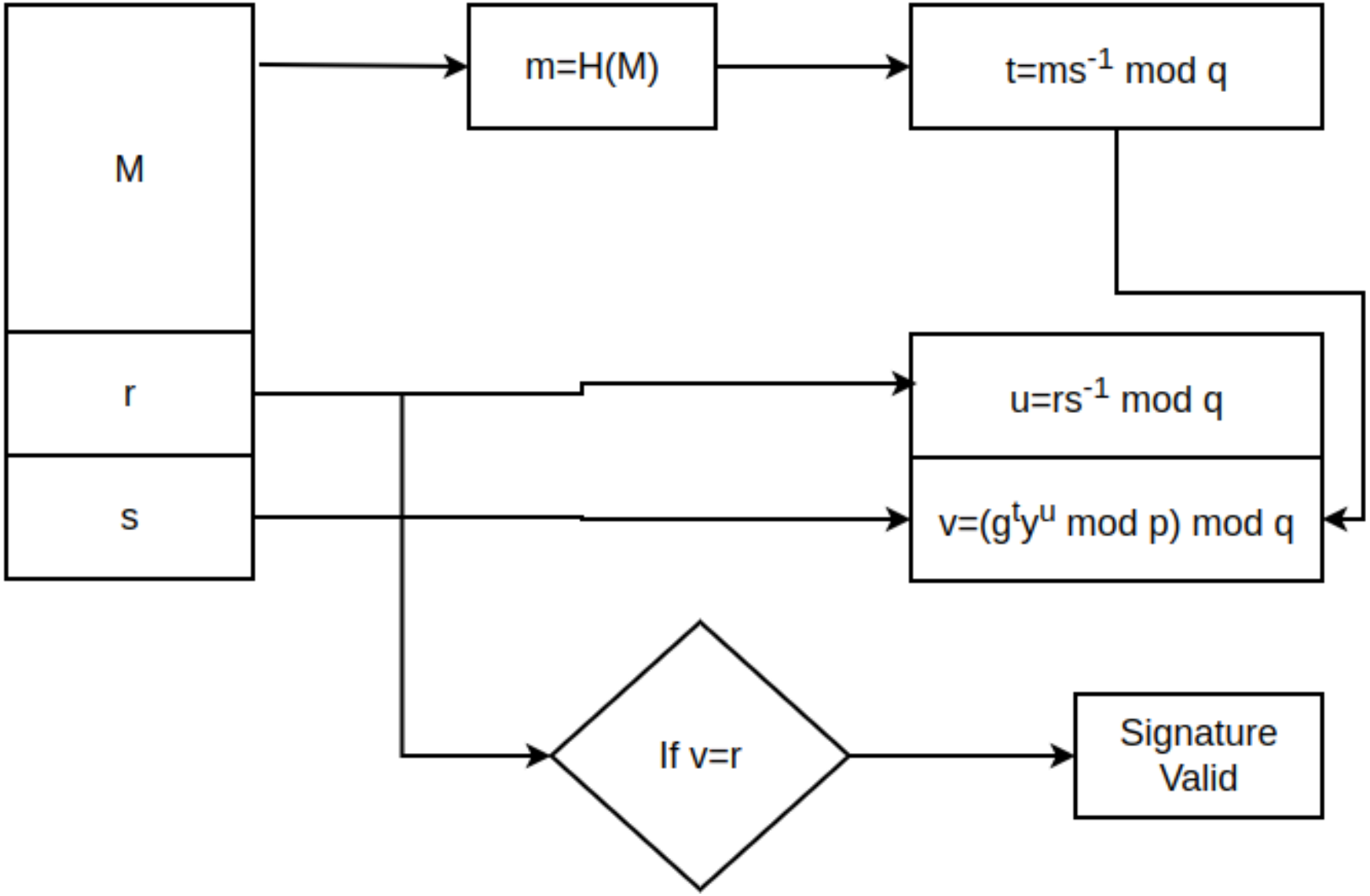
DSA: Signing



DSA: Verifying

- $t=ms^{-1} \bmod q$
- $u=rs^{-1} \bmod q$
- $v=(g^t y^u \bmod p) \bmod q$
- If v=r, then signature is valid

DSA: Verifying



DSA: Verifying

- $s^{-1} \bmod q = k(m+xr)^{-1} \bmod q$
- Or $s^{-1} (m+xr) \bmod q = k \bmod q$
- $v = (g^t y^u \bmod p) \bmod q$
- $= (g^{ms-1 \bmod q} y^{rs-1 \bmod q} \bmod p) \bmod q$
- $= (g^{ms-1 \bmod q} g^{xrs-1 \bmod q} \bmod p) \bmod q$
- $= (g^{(m+xr)s-1 \bmod q} \bmod p) \bmod q$
- $= (g^{k \bmod q} \bmod p) \bmod q$
- $= (g^k \bmod p) \bmod q$
- $= r$

DSA: Example

- p=1248563, q=101, h=2
- $g = h^{(p-1)/q} \bmod p = 2^{12362} \bmod 1248563 = 588634$
- Private key x=23
- Public key, $y = g^x \bmod p = 702644$
- H(M)=53, k=42
- $r = (g^k \bmod p) \bmod q = 52$
- $s = k^{-1}(m+xr) \bmod q = 61$

DSA: Example

- $588634^{42} \bmod 1248563$
- 42 in binary 101010 which is 32, 8 2
- $588634^{32}\ 588634^{8}\ 588634^{2} \bmod 1248563$
- $588634^{2} \bmod 1248563 = 19263$
- $588634^{4} \bmod 1248563 = 19263^{2} \bmod 1248563$
- $= 239958$
- $588634^{8} \bmod 1248563 = 239958^{2} \bmod 1248563$
- 1110456

DSA: Example

- $588634^{16} \bmod 1248563 = 1110456^{2} \bmod 1248563$
- 495061
- $588634^{32} \bmod 1248563 = 495061^{2} \bmod 1248563$
- $=1216762$
- Now, $588634^{32}\ 588634^{8}\ 588634^{2} \bmod 1248563$
- $=1216762 \times 1110456 \times 19263 \bmod 1248563$
- $=744636 \times 19263 \bmod 1248563$
- $=431524$

DSA: Example

- 431524 mod 101
- =52

DSA: Example

- $t=H(m)s^{-1} \bmod q=82$
- $u=rs^{-1} \bmod q = 29$
- $v=(g^t y^u \bmod p) \bmod q =52$
- Since v=r, then message is valid